

Unit 3 Practice Test

Practice 3.3

We are all habitual writers; our word choice and the way we phrase things are highly individualistic. Write a program to count the frequency of a given letter in a file.

Example

Given a file that contains the sample solution for this problem, count the frequency of usage of a given letter in the file.

User input is underlined.

```
What is the name of the file: /home/cs124/tests/practice33.cpp  
What letter should we count: t  
There are 125 t's in the file
```

Another execution on the same file:

```
What is the name of the file: /home/cs124/tests/practice33.cpp  
What letter should we count: i  
There are 66 i's in the file
```

Assignment

Write the program to:

- Prompt the user for the filename
- Read the data from the file one letter at a time
- Compare each letter against the target letter
- Display the count of instances on the screen
- Use proper modularization of course.

Please use the following test bed to validate your answers:

```
testBed cs124/practice33 practice33.cpp
```

You can validate your answers against:

```
/home/cs124/tests/practice33.cpp
```

Continued on the next page

Grading for Test 3

Sample grading criteria:

	Exceptional 100%	Good 90%	Acceptable 70%	Developing 50%	Missing 0%
Syntax of the array 30%	All references of the array are elegant and optimal	Array correctly declared and referenced	One bug exists	Two or more bugs	An array was not used in the problem
File interface 30%	Solution is elegant and efficient	All the data is read and error checking is performed	Able to open the file and read from it	Elements of the solution are present	No attempt was made to open the file
Problem solving 20%	Solution is elegant and efficient	Zero test bed errors	There exist only one or two flaws in the approach to solve the problem	Elements of the solution are present	Input and output do not resemble the problem
Modularization 10%	Functional cohesion and loose coupling is used throughout	Zero bugs with function syntax but there exist modularization errors	One bug exists in the syntax or use of a function	Two or more bugs	All the code exists in one function
Programming Style 10%	Well commented, meaningful variable names, effective use of blank lines	Zero style checker errors	One or two minor style checker errors	Code is readable, but serious style infractions	No evidence of the principles of elements of style in the program

Unit 3 Project : MadLib

Write a program to implement Mad Lib®. According to Wikipedia,

Mad Libs is a word game where one player prompts another for a list of words to substitute for blanks in a story; these word substitutions have a humorous effect when the resulting story is then read aloud.

The program will prompt the user for the file that describes the Mad Lib®, and then prompt him for all the substitute words. When all the prompts are complete, the program will display the completed story on the screen.

This project will be done in three phases:

- Project 08 : Design the MadLib program
- Project 09 : Read the file and display all the prompts to the user
- Project 10 : Display a MadLib for a given file and set of user input

Interface Design

The program will prompt the user for the filename of his Mad Lib®, allow him to play the game, then ask the user if he/she wants to play another. Consider the following Mad Lib® with the filename `madlibZoo.txt`:

```
Zoos are places where wild :plural_noun are kept in pens or cages :! so
that :plural_noun can come and look at them :. There is a zoo :! in the park
beside the :type_of_liquid fountain :. When it is feeding time, :! all
the animals make :adjective noises. The elephant goes :< :funny_noise
:> :! and the turtledoves go :< :another_funny_noise :. :> My favorite
animal is the :! :adjective :animal :, so fast it can outrun a/an
:another_animal :. :! You never know what you will find at the zoo :.
```

An example of the output is:

Please enter the filename of the Mad Lib: madlibZoo.txt

Plural noun: boys

Plural noun: girls

Type of liquid: lemonade

Adjective: fuzzy

Funny noise: squeak

Another funny noise: snort

Adjective: hungry

Animal: mouse

Another animal: blue-fin tuna

Zoos are places where wild boys are kept in pens or cages
so that girls can come and look at them. There is a zoo
in the park beside the lemonade fountain. When it is feeding time,
all the animals make fuzzy noises. The elephant goes "squeak"
and the turtledoves go "snort." My favorite animal is the
hungry mouse, so fast it can outrun a/an blue-fin tuna.
You never know what you will find at the zoo.

Do you want to play again (y/n)? n

Thank you for playing.

Note that there is a tab before each of the questions (ex: "Plural noun:")

File Format

Consider the following user's file called `madLibExample.txt`:

```
this is one line with a newline at the end. :!  
Here we have a comma :, and a period :. :!  
We can have :< quotes around our text :> if we want :!  
This will prompt for :< My favorite cat :> is :my_favorite_cat :. :!
```

Notice the following traits of the file:

- Every word, keyword, or punctuation is separated by a space or a newline. These are called tokens.
- Tokens have a colon before them. They are:

Symbol	Meaning
!	Newline character. No space before or after.
<	Open double quotes. No space after.
>	Close double quotes. No space before.
.	Period. No space before.
,	Comma. No space before.
<i>anything else</i>	A prompt

- If a prompt is encountered, convert the text inside the prompt to a more human-readable form. This means:
 1. Sentence-case the text, meaning capitalize the first letter and convert the rest to lowercase.
 2. Convert underscores to spaces.
 3. Precede the prompt with a tab.
 4. Put a colon and a space at the end.
 5. The user's response to the text could include spaces.

Your program will not need to be able to handle files of unlimited length. The file should have the following properties (though you will need to do error-checking to make sure):

- There are no more than 1024 characters total in the file.
- There are no more than 32 lines in the file.
- Each line has no more than 80 characters in it.
- There are no more than 256 words in the file.
- Each word is no more than 32 characters in length.

Hint: to see how to declare and pass an array of strings, please see page 227.

Hint: when displaying the story, you will need to re-insert spaces between each word. Either the word before or the word after a given space negotiate whether there is a space between the words. This means that either word can remove the space.

Project 08

The first part of the project is the design document. This consists of three parts:

1. Create a structure chart describing the entire Mad Lib® program.
2. Write the pseudocode for the function `readFile` a function to read the Mad Lib® file into some data structure (examples: a string, and array of something). You will need to include the logic for reading the entire story into the data-structure and describe how the story will be stored.
3. Write the pseudocode for the function `askQuestion`. This need to describe how to turn `":grandma's_name"` into `"\tGrandma's name: "` and also describe how to put the user's response back into the story. If, for example, the file had the tags `":favorite_car"` and `":first_pet's_name"` then the following output would result:

```
Favorite car: Ariel Atom 3  
First pet's name: Midnight
```

On campus students are required to attach [this rubric](#) to your design document. Please self-grade.

Project 09

The second part of the Mad Lib project (the first part being the design document due earlier) is to write the code necessary read the Mad Lib from a file and prompt the user:

```
Please enter the filename of the Mad Lib: madlibZoo.txt
Plural noun: boys
Plural noun: girls
Type of liquid: lemonade
Adjective: fuzzy
Funny noise: squeak
Another funny noise: snort
Adjective: hungry
Animal: mouse
Another animal: blue-fin tuna
```

Note that there is a tab before each of the questions (ex: "Plural noun:"):

Hints

- Your program will not need to be able to handle files of unlimited length. The file should have the following properties (though you will need to do error-checking to make sure):
- There are no more than 1024 characters total in the file.
- There are no more than 32 lines in the file.
- Each line has no more than 80 characters in it.
- There are no more than 256 words in the file.
- Each word is no more than 32 characters in length.

Hint: To see how to declare and pass an array of strings, please see Chapter 3.0 of the text.

Assignment

Perhaps the easiest way to do this is in a four-step process:

1. Create the framework for the program using stub functions based on the structure chart from your design document.
2. Write each function. Test them individually before "hooking them up" to the rest of the program. You are not allowed to use the String Class for this problem; only c-strings!
3. Verify your solution with testBed:

```
testBed cs124/project09 project09.cpp
```

4. Submit it with "Project 09, Mad Lib" in the program header.

An executable version of the project is available at:

```
/home/cs124/projects/prj09.out
```

Project 10

The final part of the Mad Lib project is to write the code necessary to make the Mad Lib appear on the screen:

```
Please enter the filename of the Mad Lib: madlibZoo.txt
Plural noun: boys
Plural noun: girls
Type of liquid: lemonade
Adjective: fuzzy
Funny noise: squeak
Another funny noise: snort
Adjective: hungry
Animal: mouse
Another animal: blue-fin tuna

Zoos are places where wild boys are kept in pens or cages
so that girls can come and look at them. There is a zoo
in the park beside the lemonade fountain. When it is feeding time,
all the animals make fuzzy noises. The elephant goes "squeak"
and the turtledoves go "snort." My favorite animal is the
hungry mouse, so fast it can outrun a/an blue-fin tuna.
You never know what you will find at the zoo.

Do you want to play again (y/n)? n
Thank you for playing.
```

Hints

A few hints to make the code writing a bit easier:

- The best way to store the story is in an array of words. Thus the process of reading the story from the file removes all spaces from the story.
- By default, you insert a space before each word when you display the story. The only conditions when a space is not inserted is when the preceding character is a open quote or a newline, or when the following character is a closed quote, period, or comma. In other words, do not think about removing spaces, but rather about adding them when conditions are right.
- Your program will need to be able to play any number of Mad Lib games. The easiest way to handle this is to have a while loop in main()

Assignment

Perhaps the easiest way to do this is in a five-step process:

1. Start with the code from Project 09.
2. Fix any necessary bugs.
3. Write the code to display the Mad Lib on the screen.
4. Verify your solution with testBed:

```
testBed cs124/project10 project10.cpp
```

5. Submit it with "Project 10, Mad Lib" in the program header.

An executable version of the project is available at:

```
/home/cs124/projects/prj10.out
```